CAS Box B3 / Rover R04

ESP32-CAM Development and Progress Report

Scope: ESP32-CAM capture, wireless camera service, camera-to-main-box integration, image persistence, metadata, and testing.

Excluded: main ESP32 power, rover movement, partition, and general Wi-Fi failure investigations.

Prepared from the repository history, source changes, serial output, session events, and camera test observations.

1. Camera Goal

The camera goal was to make the enclosed ESP32-CAM operate without physical button access and provide original JPEG images wirelessly to the phone-first investigation dashboard. Each captured image had to be saved by the phone, indexed, and traceable to the investigation timeline and nearby sensor data.

ESP32-CAM
capture JPEG
→
Main ESP32
camera relay
→
Phone dashboard
save + index
→
ZIP dataset
frames + metadata

Required outcome

- Software-only camera control and capture.

- Original JPEG preservation, not browser screenshots.
- Frame IDs, timestamps, sizes, status, and failure records.
- Reliable operation while telemetry and heartbeat continue.
- Camera disconnection must not destroy already-recorded data.

2. Original Camera Role and Constraints

Area	Decision
Hardware	ESP32-CAM is a separate board from the main ESP32 and is enclosed.
Operator access	No physical camera buttons can be used; all actions must be wireless/software-controlled.
Camera output	Original JPEG frames are retained. Processed images, if later created, must be separate.
Transport	Camera transfers are kept separate from latency-sensitive rover control and heartbeat traffic.
Storage	The phone is the authoritative local storage location for frames and manifests.
Analysis boundary	No on-camera vision, pillar counting, or autonomous route decisions were added.

3. What We Implemented

3.1 CameraStation changes

- Removed the active dependency on the old laptop upload path.
- Kept the camera as a wireless snapshot server.
- Added software registration with the main box.
- Added JPEG frame serving through the camera HTTP interface.
- Changed frame acquisition toward PSRAM-aware buffering.

- Changed capture behavior to prefer `CAMERA_GRAB_LATEST` so stale buffered frames are avoided.
- Added a warm-up period of approximately 2–3 frames before normal capture.
- Enabled automatic exposure and gain controls, including AEC2 where supported.
- Added the missing `write(uint8_t)` implementation to `FrameBufferStream`, resolving the abstract-class compilation error.
- Removed unreachable code after the active camera path.

3.2 Main-box relay changes

- Replaced the blocking camera fetch in the AsyncWebServer request path.
- Moved the camera fetch into a separate FreeRTOS task.
- Added a bounded cached-JPEG buffer on the main box.
- Changed the request flow so `/camera/snapshot` can return `202` while capture is pending.
- Added a cached-frame endpoint so the phone can poll for completion.
- Added JPEG start/end validation using the expected `FF D8` and `FF D9` markers.
- Avoided the previous large String/lambda copy path for image transfer.

4. Phone Dashboard Contributions

- Separated heartbeat timing from telemetry polling.
- Camera status is polled less frequently than the heartbeat.
- Camera capture is not awaited inside the heartbeat path.
- Images are treated as saved files/blobs, not merely as a displayed preview.
- Frame metadata is intended to record frame ID, filename, capture/receipt timing, byte count, status, and missing-frame events.
- The phone stores camera data in the investigation record and includes it in ZIP finalization.
- A wake lock was added to reduce timer throttling during a run.
- Camera failure events are recorded separately from rover and telemetry failures.

Contribution: the camera path was redesigned from a synchronous preview-style request into an asynchronous capture, cache, poll, validate, and persist workflow.

5. Problems We Observed

Observed issue	Evidence	Likely mechanism
Images requested but missing	Session events showed repeated `IMAGE_CAPTURE_REQUESTED` immediately followed by `IMAGE_MISSING`.	The dashboard request completed before a usable frame reached the phone, and the old path did not provide robust asynchronous completion.
Camera Wi-Fi retries	Serial output reported inability to join `CAS-Box-04` and repeated retries.	Registration/reconnect was synchronous and vulnerable to timing and link interruptions.
`cam_hal: FB-OVF`	Camera serial output repeatedly showed frame-buffer overflow.	Capture mode/buffer handling allowed contention or stale frame accumulation.
Dark images	Earlier saved images were visibly dark, especially in low light.	Exposure/gain had not been explicitly stabilized or reported.
Camera compile failure	`FrameBufferStream` could not be instantiated because `Print::write(uint8_t)` was pure virtual.	The custom Stream subclass was incomplete.
Main relay blocking	Previous relay used an HTTP call and `getString()` in an AsyncWebServer handler.	Long camera operations could stall other dashboard endpoints.

6. Failed Attempts and Corrections

Attempt	Result	Correction
Use browser-visible image as	Failed as a data-integrity	Added explicit frame saving,

proof of recording	method; preview did not prove persistence.	metadata, and missing-frame events.
Perform camera fetch directly during an async web request	Failed because the request could block while fetching a large JPEG.	Moved capture to a background task and served a cached result.
Use the incomplete FrameBufferStream	Compilation failed.	Implemented the required byte-write method and retained explicit binary transfer.
Use buffered capture behavior that could return stale frames	Associated with FB-OVF/stale-frame risk.	Changed toward latest-frame acquisition and warm-up.
Register/reconnect synchronously in the camera loop	Could interfere with serving requests and reconnection stability.	Registration/reconnect was moved conceptually off the request path.
Capture without exposure metadata	Made dark-frame diagnosis difficult.	Enabled auto exposure/gain and added the requirement to report brightness/exposure metadata.

7. Progress by Milestone

Milestone	Status	Meaning
Camera source compiles after Stream fix	Achieved	The abstract-class error was corrected.
Camera configured for latest frames and warm-up	Implemented in source	Hardware soak still required.
Automatic exposure/gain enabled	Implemented in source	Brightness behavior still needs measured capture series.
Phone-only software operation	Implemented in	No physical camera interaction is

	architecture	required by the intended workflow.
Non-blocking main-box camera relay	Implemented in source	Requires live end-to-end validation.
Fresh JPEG received and persisted	Not proven	Earlier logs still contained IMAGE_MISSING events.
100-capture soak at 3–5 s cadence	Not completed	No measured success rate is available.
95% valid image gate	Not demonstrated	Must be measured from actual saved JPEGs.
Complete camera frame manifest reconciliation	Not demonstrated	Needs a successful capture run and ZIP inspection.

8. Camera Data Model We Were Working Toward

Each image must be more than a file. It must be a traceable camera record.

Field	Purpose
frame_id	Unique identity for the captured frame.
filename	Original JPEG filename stored by the phone.
capture_timestamp	Camera-side capture time where available.
receipt_timestamp	Phone-side arrival time.
sequence_number	Ordering and gap detection.
bytes	Confirms the received file is not empty/truncated.

jpeg_valid	Validation of JPEG start/end markers and readability.
camera_status	Online, pending, retry, missing, or failed state.
nearest_sensor_sequence	Links the image to nearby LiDAR/IMU/GPS/ultrasonic data.
exposure/brightness metadata	Allows dark or overexposed frames to be diagnosed later.
event/segment reference	Connects the frame to notes, route segments, and operator decisions.

9. What Worked and What Did Not

Worked or materially improved

- The camera compilation blocker was identified and corrected.
- The camera architecture was clarified as a separate wireless snapshot server.
- The old blocking relay design was identified as unsafe for heartbeat-sensitive operation.
- The source was changed toward asynchronous capture and cached-frame serving.
- Frame freshness, PSRAM use, warm-up, exposure, gain, and JPEG validation were addressed in code.
- The phone workflow was aligned with persistent image storage and frame manifests.

Not yet proven

- Stable wireless registration and reconnect under the actual enclosed hardware.
- Successful image transfer through the complete camera → main box → phone path.
- Absence of FB-OVF during a sustained run.
- Brightness improvement under the real outdoor/indoor lighting conditions.
- 95% or better valid-frame rate.
- Every frame’s sensor/event linkage in a completed ZIP.

Important: source implementation progress is not the same as hardware acceptance. The camera work is substantially implemented, but the required capture soak and end-to-end

evidence remain outstanding.

10. Next Camera Tests

1. Start the enclosed camera and verify software-side registration only; no button access required.
2. Request one capture and confirm the phone receives a non-empty JPEG with valid `FF D8 ... FF D9` markers.
3. Repeat at least 20 times while telemetry and heartbeat polling continue.
4. Run the full 100-capture soak at the planned 3–5 second cadence.
5. Record requested, received, valid, missing, retried, stale, and corrupted counts.
6. Record frame age, byte sizes, camera status, and exposure/brightness metadata.
7. Check for FB-OVF and camera reconnect messages in serial output.
8. Export the investigation ZIP and verify every saved JPEG has exactly one manifest row.
9. Check that missing frames are explicitly reported rather than silently omitted.
10. Only after these tests, tune cadence, JPEG quality, exposure, and retry limits.

11. Final Camera Status

The project contribution this week was the design and implementation of a phone-first, software-controlled ESP32-CAM capture pipeline. We investigated the original failures, corrected the camera compilation issue, addressed stale/overflow-prone capture behavior, enabled automatic exposure controls, separated image transfer from heartbeat-sensitive requests, and designed traceable image persistence.

The main remaining camera objective is empirical validation: prove that fresh original JPEGs can be captured repeatedly, transferred wirelessly, saved by the phone, and reconciled with the frame manifest and sensor timeline. No camera vision or autonomous pillar/route logic was added.

Generated from the repository source, commits, camera serial observations, session events, and implementation history available in the workspace.